Лабораторна 2
Токенізатори

Конспект із розбором коду та відповідями на питання

Роман Миджин · 23 жовтня 2026

Я порівнюю способи, якими текст перетворюється на токени: слова, символьні n-грами, BPE та WordPiece. Потім дивлюся, скільки токенів вони створюють для англійської, української та грузинської, і порівнюю з готовими кодуваннями GPT.

1. Текст, токен, ID та embedding

Токен - одиниця розбиття: слово, частина слова, число або знак. **ID** - її номер у словнику. Номер не є значенням слова: сусідні ID не означають схожість. **Embedding** - навчений вектор, який модель бере за цим ID.

Шлях даних: **текст** → **токени** → **ID** → **вектори** → **трансформер**. У лабі я реалізую токенізацію й кодування, а не навчаю мовну модель.

```
def word_tokenize(text):
    return re.findall("[^\\W\\d_]+(?:['’][^\\W\\d_]+)*|\\d+|[^\\w\\s]|_",
        text.lower())
```

Регулярний вираз окремо знаходить слова з внутрішніми апострофами, числа та знаки. **lower()** зменшує кількість варіантів, але стирає регістр: «Роман» і «роман» перестають відрізнятися.

Як користуватися конспектом

Питання йдуть у порядку notebook: слова та словник, n-грами, BPE, WordPiece, FLORES, GPT, fertility, порівняння та змішані приклади. Для кожного підходу є переваги й недоліки. У сірих блоках - код із notebook або явно позначені окремі приклади. Довгі рядки перенесено лише для друку.

Що означає навчити токенізатор?

Я визначаю словник або правила злиття за корпусом. Це не навчання GPT чи embedding-векторів. Готові правила потім застосовую до нового тексту без повторного навчання.

2. Словник, невідомі слова та пам’ять

Послівна токенізація. Я беру ціле слово за основну одиницю та призначаю йому ID. Це найпростіший підхід: знайомі слова записуються коротко, але нові слова й словоформи потребують окремих записів або замінюються на <unk>.

```
def build_word_vocab(texts):
    tokens = sorted({token for text in texts for token in
        word_tokenize(text)})
    return {"<pad>": 0, "<unk>": 1, **{token: i for i, token in
        enumerate(tokens, 2)}}

def encode_words(text, vocab):
    return [vocab.get(token, vocab["<unk>"]) for token in word_tokenize(text)]
```

Що робить кожна частина

set прибирає повтори, **sorted** задає сталий порядок. Через **enumerate(tokens, 2)** звичайні токени отримують ID від 2. Перші два місця зайняті: <pad> = 0 і <unk> = 1.

vocab.get(token, vocab["<unk>"]) повертає ID відомого токена, а для невідомого - 1. <pad> потрібен для доповнення послідовностей до спільної довжини; він не означає невідоме слово.

OOV: out of vocabulary

Якщо навчання бачило «кіт», але не «котик», друге слово стає <unk>. Два різні невідомі слова отримають один ID. Наприклад, невідомі імена в реченнях можуть зникнути в однаковому наборі чисел.

Послівний метод дає коротке представлення знайомих слів, але погано працює з новими формами. «Студент», «студента», «студентами» займають три окремі записи. Підслова дозволяють використовувати частини повторно.

Чому великий словник потребує пам’яті

Embedding-таблиця має **V × d** параметрів: V - кількість токенів, d - довжина вектора. За V = 2000 і d = 128 це 256 000 параметрів; у float32 - приблизно 1 МБ лише для цієї таблиці, без градієнтів та станів оптимізатора.

Компроміс: більший словник може скоротити послідовність, але збільшує таблицю векторів. Менший словник економить місце, проте текст частіше розбивається на дрібні частини.

Які переваги й недоліки послівного методу?

Переваги: простий код, зрозумілі одиниці, короткі послідовності для відомих слів. **Недоліки:** великий словник через словоформи; нові слова стають <unk>; нормалізація стирає регістр. Це зручно для простого обмеженого словника, але гірше для відкритого багатомовного тексту.

3. N-грами, Жаккар і слово «кіт»

Символьні n-грами. Я ділю слово на всі сусідні фрагменти довжини n з перекриттям. На відміну від цілих слів, частини можна використати в нових словах. Довжину фрагмента задаю наперед; алгоритм її не навчає.

```
def char_ngrams(word, n=3):
    if type(n) is not int or n <= 0:
        raise ValueError("n має бути додатним цілим числом")
    if not word:
        return []
    bounded = ("^" + word + "$").ljust(n, "~")
    return [bounded[i : i + n] for i in range(len(bounded) - n + 1)]

def ngram_overlap(word_a, word_b, n=3):
    a, b = (set(char_ngrams(word_a, n)), set(char_ngrams(word_b, n)))
    return len(a & b) / len(a | b) if a | b else 1.0
```

n-грама - фрагмент із n сусідніх символів. Я додаю ^ і $ для меж слова. Для «кіт», n = 3: **^кі, кіт, іт$**. Фрагменти перекриваються; одна літера може входити в кілька n-грам.

Без padding для непорожнього слова довжини L число вікон: **max(0, L + 3 - n)**. Два маркери додають 2, а вікно має «довжина рядка - n + 1» положень. З поточним мінімальним padding число вікон - **max(1, L + 3 - n)**. Порожній рядок обробляється окремо й дає [].

Шестиграми для «кіт»

«^кіт$» має 5 символів. Без доповнення при n = 6 вікон немає й результат був би []. У поточному коді ljust додає один ~: отримую ^кіт$~ і одну шестиграму. Для n = 7 додаються два ~. Це padding слова для n-грам, а не вирівнювання батчу.

```
bounded = bounded.ljust(n, "~")
```

У notebook padding уже реалізовано перед створенням вікон. ~ - один умовний службовий символ, а не число 0 і не вбудована команда Python. Детальніше - на наступній сторінці.

Жаккар = |A ∩ B| / |A □ B|: спільні унікальні фрагменти / всі різні. Множини не зберігають порядок і повтори. 0 - нічого спільного, 1 - однакові множини. Дві порожні множини код теж оцінює як 1, тому це може приховати невдале розбиття коротких слів.

Менше n - частіші спільні частини, але менше специфічності. Більше n - більше контексту, але гірше покриття коротких і нових слів.

4. Padding: точне правило та крайові випадки

У поточній char_ngrams коротке непорожнє слово вже доповнюється до n. Нижче - окремий запис тієї самої ідеї з назвою padded_ngrams; у notebook функція називається char_ngrams.

```
def padded_ngrams(word, n=3):
    if not isinstance(n, int) or isinstance(n, bool) or n <= 0:
        raise ValueError("n має бути додатним цілим")
    if not word:
        return []
    bounded = "^" + word + "$"
    bounded = bounded.ljust(n, "~")
    return [bounded[i:i+n]
            for i in range(len(bounded) - n + 1)]
```

Слово та n	Після меж і padding	Результат
кіт, n=3	^кіт$	^кі, кіт, іт$
кіт, n=6	^кіт$~	Одна шестиграма
кіт, n=7	^кіт$~~	Одна семиграма
кіт, n=8	^кіт$~~~	Одна восьмиграма

Кількість доданих елементів: **p = max(0, n - (L + 2))**, де L - довжина непорожнього слова. Після мінімального padding маю хоча б одне положення вікна. Для довгого слова нічого не додаю.

Що важливо уточнити

Один символ, а не напис. Рядок <pad> має п’ять символів Python. Якщо просто дописати його, символьне вікно почне різати й ці літери. У прикладі ~ є одним елементом і має бути зарезервованим: якщо він дозволений у вхідному тексті, потрібна інша домовленість або екранування.

Дві різні задачі padding. Тут я подовжую слово для n-грам. Padding батчу доповнює вже готові послідовності ID до спільної довжини; там зазвичай потрібна маска, щоб модель ігнорувала доповнення. Це не одна операція.

Однакове правило. Якщо навчання не бачило padded-фрагментів, додавання padding лише при застосуванні не гарантує відсутності OOV. Також треба явно визначити порожній рядок: у прикладі він лишається [], щоб не вигадувати слово з одних маркерів.

Це мінімальне праве доповнення до n. Додавання n-1 маркерів з обох боків - інша схема: вона створює більше прикордонних вікон. Не слід перемикатися між схемами без перенавчання словника.

5. Що саме означає n в n-грамах?

У notebook: char_ngrams і ngram_overlap, перед переходом до BPE. Тут зібрані короткі питання, які випливають безпосередньо з назви методу.

Що таке n-грама?

Це послідовність із n сусідніх одиниць. n задає довжину фрагмента, а не число всіх токенів у слові чи розмір словника. Одиницею може бути символ, слово або інший токен; у цій лабораторній я беру символи Python-рядка.

Назва	n	Для ^кіт$
Уніграми	1	^, к, і, т, $
Біграми	2	^к, кі, іт, т$
Триграми	3	^кі, кіт, іт$
Шестиграми	6	Без padding немає жодного вікна

Чим символьна n-грама відрізняється від словесної?

Для «кіт спить вдома» словесні біграми - «кіт спить» і «спить вдома». Символьна біграма - дві сусідні літери або інші символи обраного рядка. У моєму char_ngrams спочатку передається окреме слово, тому фрагменти через пробіли між словами не будуються.

Чому є перекриття?

Вікно рухається на 1 позицію: ^кі, кіт, іт$. Спільні літери входять у кілька фрагментів. Якщо рухати вікно на n, це вже неперекривні блоки, а не всі сусідні n-грами. При set зберігаються лише різні фрагменти: їхні частоти та початковий порядок губляться.

Для чого ^ і $?

Вони відрізняють частини на початку, усередині та наприкінці слова. Це межі, а не padding і не букви початкового тексту. Для непорожнього слова L без маркерів число вікон - max(0, L-n+1), із двома маркерами - max(0, L+3-n).

Які переваги й недоліки n-грам?

Переваги: простота, повторне використання частин нових слів, можливість виміряти схожість. **Недоліки:** багато фрагментів через перекриття, фіксоване n не підлаштовується під частоти; для коротких слів потрібний padding; невідомі фрагменти все одно можливі. Набір n-грам із Жаккаром не є повним збереженням тексту.

Python len() рахує Unicode-кодові точки, а не байти й не завжди видимі знаки. Emoji або літера з комбінованим наголосом можуть складатися з кількох елементів.

6. BPE: частоти, злиття, нові слова

BPE. Це підсловний метод: я починаю із символів і зливаю найчастіші пари. На відміну від n-грам, довжини частин різні, вони не перекриваються, а розбиття підлаштовується під частоти навчального корпусу.

```
def most_frequent_pair(vocabulary):
    counts = Counter()
    for symbols, frequency in vocabulary.items():
        for pair in zip(symbols, symbols[1:]):
            counts[pair] += frequency
    return min(counts, key=lambda pair: (-counts[pair], pair)) if counts else
        None

def apply_bpe(word, merges):
    if not word:
        return []
    symbols = tuple(word) + ("</w>",)
    for pair in merges:
        symbols = merge_pair(symbols, pair)
    return list(symbols)
```

У train_bpe я починаю зі списків символів із </w>, рахую пари, зливаю найчастішу й повторюю. **frequency** враховує повторення слова. Ключ **(-counts[pair], pair)** вибирає найбільшу частоту через min; при рівності - пару за лексикографічним порядком.

Приклад

Нехай «кіт» є двічі, «кит» - один раз. Пара (т, </w>) трапляється 3 рази, (к, і) - 2. Перша стає токеном т</w>. Потім перераховую пари вже в змінених послідовностях. Це навчання за підрахунками, без градієнтного спуску.

У **merge_pair** і += 2 пропускає обидві злиті частини, тому входження не перекриваються. Для «ааа» і пари (а, а) результат - [аа, а]. Навчальний код обробляє кожне слово окремо; </w> позначає його кінець.

train_bpe зберігає порядок злиттів. apply_bpe повторює ці правила для нового слова. Історія важлива: раннє правило може створити частину для пізнішого. Нове слово з відомих символів можна подати дрібнішими фрагментами.

Мій BPE символьний і повертає рядки: невідомий символ може залишитися в результаті поза навчальним словником. Це не готовий механізм ID для будь-якого тексту. Byte-level BPE використовує початкові 256 байтів, щоб покрити довільний UTF-8 текст.

7. BPE: скільки разів можна зливати пари

При навчанні я задаю бюджет num_merges. Алгоритм виконує не більше стількох кроків і може завершитися раніше, якщо сусідніх пар уже немає. Готову модель можна застосовувати до будь-якої кількості текстів: це не витрачає злиття й не перенавчає словник.

Один крок - не одне входження

Один крок вибирає **один тип пари**, а merge_pair зливає всі його неперекривні входження в кожному слові. Якщо пара є тисячу разів, це все ще одне правило у merges. В одному слові кілька входжень можуть скоротитися за один крок.

```
("м", "а", "м", "а", "</w>")
# після одного правила ("м", "а"):
("ма", "ма", "</w>")
```

До злиття - 5 частин, після - 3. Наступні кроки можуть злити «ма» + «ма», а потім «мама» + </w>. Для цього прикладу слово стає одним токеном за 3 правила, хоча спочатку мало 5 частин.

Де природна межа

Для одного слова з K початкових частин потрібно не більше K-1 успішних послідовних скорочень, щоб отримати одну частину. Загальний бюджет для корпусу залежить від усіх різних слів і спільності пар. Якщо S0 - сума довжин початкових послідовностей різних слів, U - їх кількість, то S0-U є верхньою межею кількості успішних кроків у цій схемі: кожен крок скорочує хоча б одну позицію. Зазвичай кроків менше, бо правило спрацьовує багато разів.

Бюджет словника

У лабораторній початковий розмір V0 - число різних символів плюс </w>. Для цілі V я задаю **M = V - V0**. Наприклад, якщо V0 = 100, для V = 500 беру 400 злиттів. Це умовний приклад, а не виміряний алфавіт цієї роботи.

«Одне злиття додає один токен» - корисне спрощення. Якщо результат уже є в наборі або пари закінчилися, кількість унікальних токенів може не зрости до цілі. Тому фактичний словник варто перевіряти через set. Історія правил, поточне розбиття навчальних слів і повний словник - різні об’єкти.

8. BPE: розбір навчального циклу

```
def train_bpe(texts, num_merges):
    if type(num_merges) is not int or num_merges < 0:
        raise ValueError("кількість злиттів має бути невід’ємним цілим
            числом")
    vocabulary = {}
    for word, count in word_frequencies(texts).items():
        vocabulary[tuple(word) + ("</w>",)] = count
    state = _training_counts(vocabulary)
    pair_counts = state[2]
    merges = []
    for _ in range(num_merges):
        if not pair_counts:
            break
        pair = min(pair_counts, key=lambda p: (-pair_counts[p], p))
        _merge_training_pair(state, pair, merge_pair)
        merges.append(pair)
    return merges
```

word_frequencies прибирає числа й пунктуацію з навчання та рахує повтори слів. Ключ vocabulary - tuple частин, значення - частота. _training_counts створює частоти й індекс слів для кожної пари; на кожному кроці я вибираю найчастішу пару.

_merge_training_pair змінює лише слова з вибраною парою та оновлює частоти. Якщо pair_counts порожній, цикл завершується. **merges.append(pair)** зберігає правило й порядок, а не готове розбиття кожного слова.

Час і повторне використання

Нехай S - початкова кількість частин у різних словах, M - число правил. Груба межа навчання - O(M×S) операцій над частинами, без урахування довжини рядків і службових витрат Python. Повтори однакових слів враховано вагами, а не копіями. Для нового слова довжини L apply_bpe робить M проходів: грубо O(M×(L+1)).

Тому великий корпус і багато злиттів можуть довго працювати й на CPU. GPU для цієї Python-реалізації сам по собі не допоможе: код не переносить підрахунки на GPU. Кеш розбиття повторних слів у notebook прискорює застосування без зміни правил.

9. Чи справді BPE скорочує послідовності?

У notebook: average_bpe_length після базових BPE-тестів.

```
def average_bpe_length(texts, merges):
    frequencies = word_frequencies(texts)
    total_words = sum(frequencies.values())
    if not total_words:
        raise ValueError("Корпус без слів")
    return (
        sum(
            (
                count * len(apply_bpe(word, merges))
                for word, count in frequencies.items()
            )
        )
        / total_words
    )
```

Що означає формула в коді?

Для кожного різного слова я множу кількість його токенів на частоту, додаю результати й ділю на загальну кількість слів. Якщо одне слово з 2 токенів є тричі, а слово з 5 токенів - один раз, середня довжина дорівнює (3×2 + 1×5)/4 = 2.75.

Частоти не можна просто ігнорувати: середнє по двох різних словах дало б (2+5)/2 = 3.5 й не врахувало б реальну кількість повторів. Для корпусу без літерних слів функція повертає помилку.

Чому ця функція не дорівнює fertility нижче?

Тут word_frequencies залишає лише слова, без чисел і пунктуації. У великому порівнянні токенізується весь текст, а знаменник fertility визначений через split(). Тому одиниці й набір даних відрізняються.

Які переваги й недоліки BPE?

Переваги: часті частини стають великими токенами; нові слова можна складати з відомих символів; розмір навчання керується числом злиттів. Недоліки: залежність від корпусу, довший вхід для погано покритих мов, витрати на навчання; мій символьний варіант не розв’язує автоматично проблему нового алфавіту.

Готові правила не гарантують лінгвістично правильних коренів чи закінчень: BPE оптимізує статистичне розбиття. Byte-level варіант забезпечує байтове покриття, але це не означає однаково короткий текст усіма мовами.

10. WordPiece: що означає score

WordPiece. Це теж підсловний метод, але в моїй схемі пара вибирається за нормованим score, а не лише за частотою. При застосуванні я шукаю найдовші частини в готовому словнику, тоді як BPE відтворює порядок злиттів.

```
def wordpiece_pair_scores(vocabulary):
    token_counts, pair_counts = (Counter(), Counter())
    for symbols, count in vocabulary.items():
        for symbol in symbols:
            token_counts[symbol] += count
        for pair in zip(symbols, symbols[1:]):
            pair_counts[pair] += count
    return {
        pair: count / (token_counts[pair[0]] * token_counts[pair[1]])
        for pair, count in pair_counts.items()
    }
```

Формула

score(a, b) = freq(a, b) / [freq(a) × freq(b)]. Чисельник - частота сусідньої пари, знаменник - добуток частот її частин. Усі частоти зважені повтореннями слів.

Пара	freq(a,b)	freq(a)	freq(b)	score
A	20	100	50	0.004
B	2	2	2	0.5

BPE обрав би A: у неї більше повторень пари. Мій WordPiece обрав би B: у неї вищий score. Нормування підсилює пару, частини якої частіше пов’язані саме одна з одною відносно їхніх власних частот.

Score - показник для вибору злиття, а не ймовірність слова чи оцінка змісту. Рідкісна пара може мати високий score, але скорочувати мало позицій у всьому корпусі.

Як створюється словник

wordpiece_symbols розділяє слово як т, ##о, ##к, ##е, ##н. Без ## - початок, із ## - продовження. train_wordpiece зберігає стартові частини та [UNK], потім додає злиття з найбільшим score. Старі фрагменти залишаються доступними для нових слів.

Це навчальна схема з лабораторної. У курсі Hugging Face таку схему описано як реконструкцію навчання WordPiece за літературою; її не слід називати точним оприлюдненим алгоритмом навчання оригінального BERT. [1]

11. WordPiece: як розбирається нове слово

```
def apply_wordpiece(word, vocab, unk_token="[UNK]"):
    pieces, start = ([], 0)
    while start < len(word):
        end = len(word)
        while end > start:
            candidate = ("##" if start else "") + word[start:end]
            if candidate in vocab:
                break
            end -= 1
        if end == start:
            return [unk_token]
        pieces.append(candidate)
        start = end
    return pieces
```

start - початок ще не обробленої частини. **end** спочатку стоїть у кінці слова й рухається вліво. Перший знайдений candidate є найдовшим доступним фрагментом. Для start > 0 додаю ##.

Приклад із тесту

```
vocab = {"[UNK]", "ток", "токен", "##ен", "##ізація"}
apply_wordpiece("токенізація", vocab)
# ["токен", "##ізація"]
apply_wordpiece("токенх", vocab)
# ["[UNK]"]
```

Якщо решта не знаходиться, **return [unk_token]** відкидає й уже знайдений початок. Для порожнього слова цикл не запускається, результат - []. Це greedy longest-match: до коротшого вже вибраного префікса я не повертаюся.

При застосуванні BPE потрібна впорядкована історія злиттів. WordPiece користується фінальним словником і щоразу шукає найдовший фрагмент. Це друга різниця між методами, окрім частоти проти score при навчанні.

12. WordPiece: чому інтуїція може підвести

Більший словник і greedy

Більший словник дає більше доступних частин, але greedy не перебирає всі розбиття. Доданий довгий початок може змінити шлях і привести до невідомого продовження. Тому для кожного окремого нового слова не можна обіцяти монотонне покращення.

```
old = {"[UNK]", "a", "##bc", "##d"}
apply_wordpiece("abcd", old)
# ["a", "##bc", "##d"]

new = old | {"ab"}
apply_wordpiece("abcd", new)
# ["[UNK]"]
```

Новий словник змушує алгоритм почати з «ab». Потрібної частини ##c або ##cd немає, тому повертається [UNK]. Старий шлях a + ##bc + ##d досі існує, але повернення до нього в коді немає. Це приклад поведінки саме такого greedy-пошуку.

Три речі, які легко переплутати

не означає окремий символ тексту. Це службова позначка позиції частини. «a» на початку й «##a» всередині - різні записи словника. При злитті зайвий внутрішній ## прибирається, а позиція отриманого фрагмента зберігається.

Score не дорівнює частоті. Його знаменник змінює пріоритет. Це пояснює, чому багато нових записів можуть охоплювати небагато повторень у корпусі. Формулу не слід називати прямою ймовірністю або точним навчанням усіх промислових WordPiece.

Один [UNK] може приховувати довге слово. Тому падіння fertility без оцінки покриття не доводить покращення. У лабі цифри й пунктуація беруть участь в оцінці, але не в навчанні власних підсловних словників. Збільшення числа злиттів не робить ці умови однаковими.

Які перевірки я б показав

Відоме слово; нове слово з відомих символів; невідомий символ; порожній рядок; слово, для якого можливі кілька розбиттів. Для WordPiece я перевіряю результат цілого слова, а не лише те, що вдалося знайти на початку.

Які переваги й недоліки WordPiece?

Переваги: повторне використання підслів, явне розрізнення початку й продовження, застосування через готовий словник. **Недоліки:** greedy може пропустити інший повний шлях; невдала частина перетворює все слово на [UNK]; більше записів не гарантує сильного зниження fertility. Моя формула score може віддавати перевагу рідкісним парам.

13. FLORES: дані та навчання різних словників

У notebook: load_flores_dev, example_index, VOCAB_SIZES, створення bpe_models і wordpiece_models.

Я беру 997 паралельних речень кожною з трьох мов, разом 2991 навчальний рядок. Однакові індекси відповідають перекладам одного змісту.

Для чого однакова кількість речень?

Щоб мова не отримала перевагу лише через більше прикладів. Паралельність також контролює тему й зміст. Але однакове число речень не вирівнює слова, символи, словоформи або частку кожної мови у словнику. Це контроль умов, а не повна рівність.

Що означають 500, 1000 і 2000?

Це цільові розміри словників, а не кількість речень чи довжина токена. Для BPE bpe_num_merges віднімає початковий алфавіт і </w>, бо частина словника вже існує до злиттів. Фактичний розмір перевіряється окремо: бюджет кроків не завжди гарантує стільки ж нових унікальних записів.

Що навчається, а що залишається готовим?

Мої BPE і WordPiece навчаються на train_texts лише з літерних слів, за винятком внутрішніх апострофів. apply_bpe_to_text і apply_wordpiece_to_text потім застосовують моделі до всіх токенів речення, включно зі знаками та числами. GPT-encoding я беру вже навченим.

Чому важливо мати окремий тестовий набір?

Якщо оцінюю на тому самому dev, я бачу представлення навчального корпусу. Для оцінки нових текстів треба зафіксувати правила й перевірити інший набір без перенавчання. 997 речень не представляють усі теми та мовні явища.

Чи потрібний GPU?

Ці реалізації використовують Python-словники, Counter і цикли. Вони працюють на CPU; наявність GPU не переносить на нього код автоматично. Більший корпус і більше злиттів збільшують час; кеш повторних слів може прискорити застосування без зміни токенів.

14. Українська, англійська та кодування GPT

Готові GPT-токенізатори. Тут я вже не навчаю власні правила, а використовую готові кодування tiktoken на основі байтового BPE. Вони повертають ID, можуть подати звичайний UTF-8 текст без словесного [UNK], але дроблять різні мови з різною ефективністю.

Чому GPT-2 дає більше токенів українською

Причин кілька: словник GPT-2 краще покриває англійські фрагменти; українська має багатшу словозміну; кирилиця займає кілька байтів UTF-8, і без відповідних злиттів її запис дробиться сильніше. Одними відмінками всю різницю пояснити не можна.

Морфологія: студент, студента, студентом, студентами - різні форми. Відмінок, число, рід і узгодження збільшують різноманіття форм у корпусі. Англійська теж має число, час та інші зміни, проте її система відмінкових форм значно менша. Тому неправильно казати, що в англійській зовсім немає граматичних форм.

Письмо й байти: ASCII-літера зазвичай має 1 байт, українська - 2, грузинська - 3. Якщо в словнику бракує корисних злиттів, потрібні дрібніші фрагменти. Два байти не дорівнюють автоматично двом токенам: злиття може охопити і кілька літер, і частину символу.

Назва у лабораторній	Encoding	Що означає збіг
GPT-2	gpt2	Окреме старіше кодування
GPT-3.5 / GPT-4	cl100k_base	Однаковий encoding дає однакові ID для того самого тексту
GPT-5	o200k_base	У цьому досліді коротші послідовності для всіх трьох мов

Відповідності перевірені за поточним tiktoken та кодом лабораторної. Це конкретні імена моделей у викликах encoding_for_model, а не твердження про всі варіанти сімейств GPT. [2]

Що можна сказати про результати

GPT-2: 1.228 токена на слово англійською, але 8.024 українською й 21.545 грузинською. У цьому наборі GPT-5 має 1.219 / 2.442 / 2.780. Це показує коротше представлення цих текстів у відповідних кодуваннях.

Я не роблю висновку, що модель краще відповідає чи знає мову лише через меншу fertility. Тут немає тесту генерації: tiktoken локально повертає ID, API-запити до моделей не виконуються.

Переваги й обмеження готового GPT-кодування

Переваги: не потрібно навчати словник; байтове подання звичайного тексту не потребує словесного [UNK]; зберігається текст. **Обмеження:** словник не підлаштований саме під мій невеликий корпус; довжина різна для мов; потрібен encoding, сумісний із моделлю. Довільний власний словник не можна підставити у вже навчену GPT: ID пов’язані з її embedding-таблицею.

15. Fertility: формула й інтерпретація

Fertility - метрика, а не ще один токенізатор. Вона показує, скільки токенів у середньому потрібно на слово. Так я порівнюю довжину представлення різними методами, а збереження інформації перевіряю окремо.

У notebook: fertility, test_fertility.

```
def fertility(tokenize, texts):
    total_tokens = 0
    total_words = 0
    for text in texts:
        total_tokens += len(tokenize(text))
        total_words += len(text.split())
    if total_words == 0:
        raise ValueError("у корпусі немає слів")
    return total_tokens / total_words
```

Fertility = загальна кількість токенів / загальна кількість слів. 250 токенів на 100 слів дають 2.5. У знаменнику використовую split(): «кіт,» - одна частина між пробілами, хоча кома може стати окремим токеном.

Я ділю суми всього корпусу. Середнє fertility окремих речень дало б однакову вагу короткому й довгому реченню. Наприклад: 1 слово → 4 токени та 9 слів → 9 токенів. Корпусна fertility = 13/10 = 1.3, а середнє часток = (4 + 1)/2 = 2.5.

Що дає коротший вхід

За доброго покриття нижча fertility дозволяє вмістити більше слів у контекстне вікно. Матриця повної self-attention має розмір T × T, де T - число токенів, тому довжина впливає на витрати. Час усієї моделі не визначається лише цією матрицею.

Один [UNK] теж займає одну позицію. Отже, нижча fertility може бути наслідком втрати інформації. Я перевіряю також UNK, відновлення тексту й результат на нових даних. Для порожнього корпусу функція видає ValueError, щоб не ділити на нуль.

16. Результати: що показують числа

Числа взято зі збережених результатів актуального lab2.ipynb, виконаного повністю на FLORES dev. Для цього конспекту навчання повторно не запускалося.

Токенізатор	Англійська	Українська	Грузинська
BPE-500	3.387	4.323	4.921
BPE-1000	2.886	3.665	4.208
BPE-2000	2.469	3.185	3.563
WordPiece-500	4.827	5.945	7.307
WordPiece-1000	4.784	5.810	7.169
WordPiece-2000	4.774	5.412	7.160
GPT-2	1.228	8.024	21.545
GPT-3.5 / GPT-4	1.231	4.202	15.345
GPT-5	1.219	2.442	2.780

Чому BPE зменшує fertility сильніше

BPE зливає найчастіші пари й прибирає багато повторних позицій. Мій WordPiece за нормованим score може вибирати рідкісні фрагменти, які мало впливають на середню довжину. Тому при збільшенні словника BPE дає сильніше скорочення. Це пояснення конкретного досліду, а не універсальна властивість WordPiece.

Для української 500 → 2000: BPE 4.323 → 3.185 (приблизно -26%), WordPiece 5.945 → 5.412 (приблизно -9%). Для грузинської WordPiece 7.307 → 7.160 змінюється зовсім мало.

У збереженому запуску частка вхідних одиниць word_tokenize, що стали [UNK], дорівнювала 13.22% / 18.24% / 19.02% для трьох мов і не змінювалася з розміром словника. Навчання брало тільки слова, а оцінка - також числа й пунктуацію. Нові злиття літер ці одиниці не додають.

Свої моделі оцінено на тому самому dev, де їх навчали: це **train fertility**. GPT має інші навчальні дані, більші словники та зберігає регістр. Порівняння не ізолює лише вплив алгоритму.

17. Змішані приклади: що перевіряю після таблиці

У notebook: MIXED_CASES і compare_mixed_cases, далі підсумкове порівняння методів.

```
def compare_mixed_cases(texts, tokenizers):
    for index, text in enumerate(texts, 1):
        print(f"{index}. {text}")
    print()
    print(f"{'tokenizer':<18}" + "".join(f"{i:>7}" for i in range(1,
        len(texts) + 1)))
    for name, tokenize in tokenizers.items():
        counts = [len(tokenize(text)) for text in texts]
        print(f"{name:<18}" + "".join(f"{count:>7}" for count in counts))
```

Що саме друкує функція?

Функція нумерує речення, потім друкує таблицю: рядки - токенізатори, стовпці - номери речень, значення - повна кількість токенів. Після таблиці в notebook є два короткі розбиття для BPE-2000 і WordPiece-2000. Великих списків ID для всіх моделей немає.

Навіщо змішувати мови, числа й URL?

Середня fertility приховує окремі провали. Змішаний текст перевіряє перемикання мов; числа, email, URL, emoji й нові імена - покриття поза звичайними словами. Один [UNK] коротший за кілька підслів, але втрачає зміст.

Що означає збереження тексту?

Мій lower() стирає регістр, а word_tokenize не повертає пробіли. Навіть без [UNK] це не точне відновлення початкового тексту. У GPT окремий токен може містити неповний UTF-8 символ; відновлювати треба весь коректний потік байтів.

Як сформулювати висновок?

Я порівнюю довжину разом із покриттям і збереженням тексту. Свої моделі перевіряю на навчальному корпусі, GPT має інший словник і дані. Тому таблиця описує цей експеримент, а не доводить найкращий метод для всіх мов чи найрозумнішу мовну модель.

18. Переваги й недоліки: підсумок

Підхід	Переваги	Недоліки
Слова	Простота, зрозумілі токени, короткий запис знайомих слів	Великі словники, окремі словоформи, OOV і втрата невідомих слів
Символьні n-грами	Прості спільні частини, придатні для схожості й роботи з формами слів	Перекриття й багато елементів; фіксоване n; короткі слова; set втрачає порядок і частоти
Мій BPE	Часті частини зливаються; керований бюджет; нові слова з відомого алфавіту	Залежність від корпусу; дорогі повні перерахунки; невідомий символ поза словником
Мій WordPiece	Підслова та повторне використання; позначка продовження; готовий словник	Greedy без повернення; [UNK] цілого слова; score не мінімізує прямо число токенів
GPT / tiktoken	Готове кодування, байтове покриття, збереження звичайного тексту	Різна ефективність для мов; немає адаптації до мого корпусу; прив’язка до моделі

Який підхід найкращий?

Без задачі універсальної відповіді немає. Для простого закритого словника може вистачити слів. Для пошуку спільних фрагментів корисні n-грами. Для відкритого тексту підслова зменшують залежність від відомих цілих слів. Для готової GPT я використовую сумісний із нею encoding.

Який компроміс мають усі словники?

Більше записів потребує пам’яті й збільшує embedding-таблицю V×d, але часто скорочує послідовності. Менше записів простіше зберігати, проте нові тексти можуть дробитися сильніше. Також важливі регістр, пунктуація, швидкість та окремий тестовий набір.

19. Короткі відповіді та порівняння

Метод	Ключове правило	Обмеження
Слова	Цілий токен → ID	Нове слово → <unk>; багато словоформ
N-грами	Вікно n символів із перекриттям	Потрібний padding коротких слів; set втрачає порядок
BPE	Найчастіша пара; потім порядок злиттів	Мій символьний варіант обмежений алфавітом
WordPiece	Score при навчанні; greedy при застосуванні	Невдала частина → [UNK] цілого слова
GPT / tiktoken	Готове байтове кодування зі злиттями	Довжина залежить від мови та словника

Що таке fertility?

Середня кількість токенів на слово: ділю загальну кількість токенів на загальну кількість слів. Менше - коротший вхід, але треба перевіряти, чи це не через [UNK].

BPE і WordPiece - у чому дві різниці?

При навчанні: частота проти нормованого score. При застосуванні: історія злиттів проти найдовших фрагментів із готового словника.

Чому більше слів у словнику не завжди сильно допомагає?

Важливо, які саме фрагменти додано. Рідкісні частини мало змінюють середнє; нові злиття відомих літер не усувають усі невідомі одиниці.

«Кіт» і шестиграми?

З маркерами має 5 символів. Без padding вікно 6 не вмістилося б. Мій поточний код додає один ~ і повертає ^кіт$~. Padding батчу лише вирівнює списки ID після токенізації та не створює відсутні n-грами.

Навіщо по 997 речень?

Щоб мова не домінувала лише через число прикладів; паралельні переклади вирівнюють зміст. Слова, символи й частки словника від цього не стають однаковими.

Джерела й прив’язка до роботи

Код і таблиця: локальний lab2.ipynb станом на 23.10.2026. Короткі вступи додані до конспекту; сам notebook під час цього оновлення не змінювався.

[1] Hugging Face: WordPiece · BPE
[2] OpenAI tiktoken: відповідності моделей і encoding
[3] FLORES: корпус для експерименту